Journo — Trình Bày Dự Án Fullstack (Tiếng Việt)

Phần 1: Tổng Quan & Kiến Trúc Hệ Thống

> Giọng văn ngôi thứ nhất — dành cho phỏng vấn kỹ thuật fullstack.
> Tác giả: TuanTu | Stack: React 19 + Laravel 12 + MySQL

---

1. TỔNG QUAN DỰ ÁN

1.1 Hệ thống làm gì, giải quyết vấn đề gì cho ai?

Journo là ứng dụng blog fullstack cho phép người dùng đăng ký tài khoản, đăng nhập, viết và chia sẻ bài viết với cộng đồng. Người đọc có thể xem bài, bình luận và tương tác; người viết có toàn quyền quản lý nội dung của mình.

Vấn đề tôi giải quyết: tạo một nền tảng viết blog có đủ luồng hoàn chỉnh — từ xác thực người dùng, phân quyền CRUD, đến hệ thống bình luận phân cấp — với kiến trúc API-first tách biệt hoàn toàn giữa frontend và backend.

1.2 Stack công nghệ đầy đủ

Tầng	Công nghệ	Phiên bản
Frontend	React	19.2.5
Frontend Router	React Router DOM	7.14.2
Frontend HTTP	Axios	1.15.2
Frontend Build	Vite	8.0
Backend Framework	Laravel	12.x
Backend Runtime	PHP	8.2
Authentication	Laravel Sanctum	4.0
Database	MySQL	8.x (qua XAMPP)
Dev Server	`php artisan serve`	port 8000

1.3 Tính năng chính

1. **Đăng ký / Đăng nhập** — token-based authentication qua Laravel Sanctum
2. **Xem danh sách bài viết** — phân trang offset-based, chỉ bài `published`
3. **Xem chi tiết bài viết** — hiển thị nội dung, tác giả, danh mục, bình luận
4. **Tạo bài viết** — form đầy đủ với title, excerpt, content, status (draft/published)
5. **Chỉnh sửa bài viết** — chỉ tác giả mới được sửa (kiểm tra quyền phía backend)
6. **Xóa bài viết** — có confirm dialog, kiểm tra quyền phía backend
7. **Bình luận** — đăng bình luận khi đã login, hiển thị bình luận phân cấp (replies)
8. **Điều hướng thông minh** — Navbar tự hiện/ẩn tùy trạng thái đăng nhập

Cơ sở hạ tầng đã thiết kế nhưng chưa có UI:

- Hệ thống Like bài viết
- Hệ thống Follow người dùng
- Quản lý Media/File upload

- Phân cấp Category (parent/child)

1.4 Điểm kỹ thuật nổi bật so với CRUD app thông thường

1. **API-first architecture** — frontend và backend hoàn toàn độc lập, giao tiếp qua REST API JSON; backend không render HTML
2. **Token authentication với Sanctum** — mỗi token được lưu trong database (`personal_access_tokens`), có thể revoke từng token riêng lẻ — khác với JWT stateless
3. **Authorization ở tầng controller** — kiểm tra `post->user_id === request->user()->id` trực tiếp trong controller, ngăn người dùng A sửa bài của người dùng B
4. **Eager loading chống N+1** — dùng `Post::with('author', 'category', 'tags')` để load quan hệ trong 1 query thay vì N+1 queries
5. **Threaded comments** — bình luận có `parent_id` hỗ trợ reply lồng nhau
6. **Database schema production-ready** — đầy đủ foreign key constraints, cascade rules, unique indexes, enum types

---

2. KIẾN TRÚC HỆ THỐNG

2.1 Diagram tổng thể

```
┌──────────────────────────────────────────────────────────────────────┐
│                        BROWSER (User)                                 │
│                                                                       │
│   ┌──────────────────────────────────────────────────────────────┐   │
│   │                    React Application                         │   │
│   │                                                              │   │
│   │   ┌──────────┐   ┌──────────┐   ┌──────────────────────┐    │   │
│   │   │  Navbar  │   │  Pages   │   │   Axios HTTP Client  │    │   │
│   │   │          │   │ Home     │   │                      │    │   │
│   │   │ reads    │   │ Login    │   │  baseURL: :8000/api  │    │   │
│   │   │ token    │   │ Register │   │                      │    │   │
│   │   │ from     │   │ PostDtl  │   │  interceptor: auto   │    │   │
│   │   │localStorage│ CreatePst│   │  attach Bearer token │    │   │
│   │   │          │   │ EditPost │   │                      │    │   │
│   │   └──────────┘   └──────────┘   └──────────────────────┘    │   │
│   │                      │                     │               │   │
│   │                      │                     │               │   │
│   │              useState/useEffect             │               │   │
│   │              (local state per page)         │               │   │
│   └──────────────────────────────────────────┬──────────────────┘   │
│                                              │                       │
│                   localStorage               │ HTTP Requests         │
│                ┌──────────────┐              │ (JSON + Bearer)       │
│                │ "token": "" │               │                       │
│                └──────────────┘              │                       │
└──────────────────────────────────────────────┼───────────────────────┘
                                               │
                                               │
                                ═══════════════╪═══════════════
                                    NETWORK (HTTP/REST)
                                               │
                                               │
┌──────────────────────────────────────────────┼───────────────────────┐
│                  Laravel 12 Backend          │                       │
│                                              ▼                       │
│   ┌──────────────────────────────────────────────────────────────┐   │
```

```
│   │                  bootstrap/app.php                        │   │
│   │   withRouting → api.php | withExceptions → JSON errors    │   │
│   └────────────────────────────┬──────────────────────────────┘   │
│                                │                                   │
│                                ▼                                   │
│   ┌──────────────────────────────────────────────────────────┐   │
│   │               routes/api.php (Router Layer)               │   │
│   │                                                           │   │
│   │  Public:     POST /register, POST /login                  │   │
│   │              GET /posts, GET /posts/{id}                  │   │
│   │              GET /categories, GET /posts/{id}/comments    │   │
│   │                                                           │   │
│   │  Protected:  middleware('auth:sanctum') {                 │   │
│   │    POST /logout, GET /me                                  │   │
│   │    POST/PUT/DELETE /posts                                 │   │
│   │    POST/DELETE /comments                                  │   │
│   │  }                                                        │   │
│   └────────────────────────────┬──────────────────────────────┘   │
│                                │                                   │
│                                ▼                                   │
│   ┌──────────────────────────────────────────────────────────┐   │
│   │            Controllers (Api namespace)                    │   │
│   │                                                           │   │
│   │  AuthController    PostController    CommentController    │   │
│   │  CategoryController                                       │   │
│   │                                                           │   │
│   │  Validate → Authorize → Call Model → Return JSON          │   │
│   └────────────────────────────┬──────────────────────────────┘   │
│                                │                                   │
│                                ▼                                   │
│   ┌──────────────────────────────────────────────────────────┐   │
│   │              Eloquent Models (ORM Layer)                  │   │
│   │                                                           │   │
│   │  User   Post   Comment   Category   Tag                   │   │
│   │  Like   Follow   Media                                    │   │
│   │                                                           │   │
│   │  Relationships: hasMany, belongsTo, belongsToMany         │   │
│   └────────────────────────────┬──────────────────────────────┘   │
│                                │                                   │
└────────────────────────────────┼───────────────────────────────────┘
                                 │
                                 │
┌────────────────────────────────┼───────────────────────────────────┐
│              MySQL Database    │                                   │
│                                ▼                                   │
│  users  posts  categories  tags  post_tag                          │
│  comments  likes  follows  media                                   │
│  personal_access_tokens  (Sanctum table)                           │
└────────────────────────────────────────────────────────────────────┘
```

2.2 Diagram phân lớp Backend

```
HTTP Request
     │
     ▼
┌──────────────────────────────────────────┐
│         Sanctum Middleware               │
```

```
│  (auth:sanctum)                          │
│  - Đọc Bearer token từ header            │
│  - Tra cứu trong personal_access_tokens  │
│  - Nếu valid → bind $request->user()     │
│  - Nếu invalid → 401 JSON response       │
└──────────────────────────────────────────┘
                    │
                    │
                    ▼
┌──────────────────────────────────────────┐
│          Controller Layer                │
│  - Validate input ($request->validate)   │
│  - Authorization check (user_id match)   │
│  - Gọi Eloquent Model                    │
│  - Return response()->json(...)          │
└──────────────────────────────────────────┘
                    │
                    │
                    ▼
┌──────────────────────────────────────────┐
│          Eloquent ORM Layer              │
│  - Model::find(), Model::where()         │
│  - with() eager loading                  │
│  - Relationship traversal                │
│  - save(), delete()                      │
└──────────────────────────────────────────┘
                    │
                    │
                    ▼
┌──────────────────────────────────────────┐
│          MySQL Database                  │
│  - PDO prepared statements (via Laravel)│
│  - Foreign key enforcement               │
│  - Cascade deletes                       │
└──────────────────────────────────────────┘
```

2.3 Bảng vai trò từng file/module quan trọng

File	Vai trò	Lý do tách riêng
`frontend/src/api/axios.js`	Tạo Axios instance, gắn token interceptor	Tách ra để mọi page dùng chung 1 instance, không viết token logic lặp lại
`frontend/src/App.jsx`	Định nghĩa toàn bộ routes của ứng dụng	Entry point cho routing, giúp nhìn một chỗ thấy toàn bộ cấu trúc URL
`frontend/src/components/NavBar.jsx`	Navigation bar, đọc token từ localStorage	Component dùng chung, không phụ thuộc vào state của page
`backend/routes/api.php`	Định nghĩa API endpoints, phân nhóm public/protected	Tách routing khỏi logic, nhìn vào thấy ngay toàn bộ API surface

File	Vai trò	Lý do tách riêng
`backend/app/Http/Controllers/Api/AuthController.php`	Xử lý register/login/logout/me	Tách auth logic khỏi business logic
`backend/app/Http/Controllers/Api/PostController.php`	CRUD bài viết, có authorization check	Single Responsibility: 1 controller cho 1 resource
`backend/app/Models/Post.php`	Eloquent model, định nghĩa relationships	Model biết cách kết nối với các bảng liên quan
`backend/bootstrap/app.php`	Cấu hình app, custom exception handler	Tập trung exception handling: AuthenticationException → JSON 401
`backend/database/migrations/*.php`	Schema definition dạng code	Version-controlled schema, reproducible trên mọi môi trường

2.4 Luồng dữ liệu end-to-end

Flow 1: Authentication (Login → Token → Protected Request)

```
[User nhập email/password vào Login.jsx]
        │
        │ handleSubmit() — event.preventDefault()
        ▼
[api.post('/login', { email, password })]
        │
        │ Axios interceptor: KHÔNG gắn token (chưa có)
        ▼
[POST http://127.0.0.1:8000/api/login]
        │
        ▼
[routes/api.php] → Route::post('/login', [AuthController::class, 'login'])
        │
        ▼
[AuthController::login()]
  │  User::where('email', $email)->first()
  │  → nếu không tìm thấy: return 404
  │
  │  Hash::check($password, $user->password)
  │  → Bcrypt verify: so sánh plaintext với hash trong DB
  │  → nếu sai: return 401
  │
  │  $user->createToken('auth_token')->plainTextToken
  │  → Laravel Sanctum tạo random token, lưu hash vào personal_access_tokens
  │  → trả về plaintext (chỉ lần này duy nhất)
        │
        ▼
[Response JSON: { user: {...}, token: "1|abc123..." }]
        │
        ▼
[Login.jsx nhận token]
```

```
    localStorage.setItem("token", token)
    window.location.href = "/"   ← hard redirect (không dùng navigate())
          │
          ▼
  [Request tiếp theo — ví dụ: api.post('/posts', {...})]
          │
          ▼
  [Axios interceptor chạy]
    const token = localStorage.getItem("token")
    config.headers.Authorization = `Bearer ${token}`
          │
          ▼
  [POST /api/posts với header: Authorization: Bearer 1|abc123...]
          │
          ▼
  [Sanctum middleware 'auth:sanctum']
    - Lấy token từ header
    - Hash token → tìm trong personal_access_tokens
    - Nếu match → bind user vào $request
    - Nếu không → AuthenticationException → bootstrap/app.php bắt → JSON 401
          │
          ▼
  [PostController::store() chạy với $request->user() đã có giá trị]
```

Flow 2: Business Flow — Tạo Bài Viết

```
  [User điền form trong CreatePost.jsx]
    title, excerpt, content, status (draft/published)
          │
          │ handleSubmit() — gọi setLoading(true), setError(null)
          ▼
  [api.post('/posts', { title, content, excerpt, status })]
          │
          │ interceptor gắn Bearer token
          ▼
  [Sanctum xác thực token → $request->user() = User object]
          │
          ▼
  [PostController::store()]
    │
    │ $request->validate([...]) — Laravel validation rules
    │   'title' required|string|max:255
    │   'content' required|string
    │   → nếu fail: tự động return 422 với error messages
    │
    │ $post = new Post()
    │ $post->user_id = $request->user()->id   ← lấy từ auth, KHÔNG từ request body
    │ $post->title = $request->title
    │ $post->slug = Str::slug($request->title) ← auto-generate URL-friendly slug
    │ $post->status = $request->status ?? 'draft'
    │
    │ if ($post->status === 'published') {
    │     $post->published_at = now()         ← timestamp khi publish
    │ }
    │
```

```
  │ $post->save()                          ← INSERT INTO posts ...
  │
        ▼
[Response JSON: post object, HTTP 201 Created]
        │
        ▼
[CreatePost.jsx: window.location.href = "/" ← redirect về Home]
        │
        ▼
[Home.jsx fetchPosts()]
  api.get('/posts')
  → PostController::index()
  → Post::with('author','category','tags')
      ->where('status','published')
      ->orderBy('published_at','desc')
      ->paginate(10)
  → trả về paginated JSON
        │
        ▼
[Home.jsx render danh sách posts]
```

2.5 Database Schema Chi Tiết

```sql
-- Bảng 1: users (Authentication & Profiles)
CREATE TABLE users (
  id                BIGINT UNSIGNED AUTO_INCREMENT PRIMARY KEY,
  name              VARCHAR(255) NOT NULL,
  email             VARCHAR(255) NOT NULL UNIQUE,  -- constraint tránh duplicate email
  email_verified_at TIMESTAMP NULL,
  password          VARCHAR(255) NOT NULL,          -- bcrypt hash
  remember_token    VARCHAR(100) NULL,
  created_at        TIMESTAMP NULL,
  updated_at        TIMESTAMP NULL
);

-- Bảng 2: categories (Hierarchical)
CREATE TABLE categories (
  id          BIGINT UNSIGNED AUTO_INCREMENT PRIMARY KEY,
  name        VARCHAR(255) NOT NULL,
  slug        VARCHAR(255) NOT NULL UNIQUE,
  description TEXT NULL,
  color       VARCHAR(7) DEFAULT '#000000',         -- hex color code
  parent_id   BIGINT UNSIGNED NULL,                 -- self-referencing FK
  FOREIGN KEY (parent_id) REFERENCES categories(id) ON DELETE SET NULL,
  created_at  TIMESTAMP NULL,
  updated_at  TIMESTAMP NULL
);

-- Bảng 3: posts (Core content)
CREATE TABLE posts (
  id           BIGINT UNSIGNED AUTO_INCREMENT PRIMARY KEY,
  user_id      BIGINT UNSIGNED NOT NULL,
  category_id  BIGINT UNSIGNED NULL,
  title        VARCHAR(255) NOT NULL,
  slug         VARCHAR(255) NOT NULL UNIQUE,        -- SEO-friendly URL
  excerpt      TEXT NULL,
```

```sql
  content      LONGTEXT NOT NULL,                    -- full blog content
  cover_image  VARCHAR(255) NULL,
  status       ENUM('draft','published','archived') DEFAULT 'draft',
  published_at TIMESTAMP NULL,                       -- set khi publish
  view_count   INT UNSIGNED DEFAULT 0,
  reading_time SMALLINT UNSIGNED NULL,
  FOREIGN KEY (user_id)     REFERENCES users(id)      ON DELETE CASCADE,
  FOREIGN KEY (category_id) REFERENCES categories(id) ON DELETE SET NULL,
  created_at   TIMESTAMP NULL,
  updated_at   TIMESTAMP NULL
);

-- Bảng 4: tags
CREATE TABLE tags (
  id         BIGINT UNSIGNED AUTO_INCREMENT PRIMARY KEY,
  name       VARCHAR(255) NOT NULL,
  slug       VARCHAR(255) NOT NULL UNIQUE,
  created_at TIMESTAMP NULL,
  updated_at TIMESTAMP NULL
);

-- Bảng 5: post_tag (Junction table — N-N)
CREATE TABLE post_tag (
  post_id BIGINT UNSIGNED NOT NULL,
  tag_id  BIGINT UNSIGNED NOT NULL,
  PRIMARY KEY (post_id, tag_id),                     -- composite PK tránh duplicate
  FOREIGN KEY (post_id) REFERENCES posts(id) ON DELETE CASCADE,
  FOREIGN KEY (tag_id)  REFERENCES tags(id)  ON DELETE CASCADE
);

-- Bảng 6: comments (Threaded)
CREATE TABLE comments (
  id          BIGINT UNSIGNED AUTO_INCREMENT PRIMARY KEY,
  post_id     BIGINT UNSIGNED NOT NULL,
  user_id     BIGINT UNSIGNED NOT NULL,
  parent_id   BIGINT UNSIGNED NULL,                  -- NULL = top-level comment
  content     TEXT NOT NULL,
  is_approved BOOLEAN DEFAULT FALSE,
  FOREIGN KEY (post_id)   REFERENCES posts(id)    ON DELETE CASCADE,
  FOREIGN KEY (user_id)   REFERENCES users(id)    ON DELETE CASCADE,
  FOREIGN KEY (parent_id) REFERENCES comments(id) ON DELETE CASCADE,
  created_at  TIMESTAMP NULL,
  updated_at  TIMESTAMP NULL
);

-- Bảng 7: likes (Unique per user per post)
CREATE TABLE likes (
  id         BIGINT UNSIGNED AUTO_INCREMENT PRIMARY KEY,
  user_id    BIGINT UNSIGNED NOT NULL,
  post_id    BIGINT UNSIGNED NOT NULL,
  UNIQUE KEY unique_like (user_id, post_id),         -- 1 user chỉ like 1 lần
  FOREIGN KEY (user_id) REFERENCES users(id) ON DELETE CASCADE,
  FOREIGN KEY (post_id) REFERENCES posts(id) ON DELETE CASCADE,
  created_at TIMESTAMP NULL,
  updated_at TIMESTAMP NULL
);

-- Bảng 8: follows (Social graph)
```

```sql
CREATE TABLE follows (
  id           BIGINT UNSIGNED AUTO_INCREMENT PRIMARY KEY,
  follower_id  BIGINT UNSIGNED NOT NULL,            -- người follow
  following_id BIGINT UNSIGNED NOT NULL,            -- người được follow
  UNIQUE KEY unique_follow (follower_id, following_id),
  FOREIGN KEY (follower_id)  REFERENCES users(id) ON DELETE CASCADE,
  FOREIGN KEY (following_id) REFERENCES users(id) ON DELETE CASCADE,
  created_at   TIMESTAMP NULL,
  updated_at   TIMESTAMP NULL
);

-- Bảng 9: media (File management)
CREATE TABLE media (
  id         BIGINT UNSIGNED AUTO_INCREMENT PRIMARY KEY,
  post_id    BIGINT UNSIGNED NULL,
  user_id    BIGINT UNSIGNED NOT NULL,
  file_name  VARCHAR(255) NOT NULL,
  file_path  VARCHAR(255) NOT NULL,
  file_type  VARCHAR(50)  NOT NULL,
  file_size  BIGINT UNSIGNED NOT NULL,
  mime_type  VARCHAR(100) NOT NULL,
  FOREIGN KEY (post_id) REFERENCES posts(id) ON DELETE SET NULL,
  FOREIGN KEY (user_id) REFERENCES users(id) ON DELETE CASCADE,
  created_at TIMESTAMP NULL,
  updated_at TIMESTAMP NULL
);

-- Bảng Sanctum (auto-created bởi Laravel)
-- personal_access_tokens: lưu hash của token, user_id, tokenable_type
```

Relationships tóm tắt:

- `users` → `posts`: 1-N (một user viết nhiều bài)
- `users` → `comments`: 1-N
- `posts` → `comments`: 1-N (xóa post → cascade xóa comments)
- `posts` ↔ `tags`: N-N (qua bảng `post_tag`)
- `categories` → `categories`: self-referencing 1-N (category cha/con)
- `users` ↔ `users` (follows): N-N self-referencing

Tiếp theo: Phần 2 — Giải thích sâu từng thành phần kỹ thuật